

分布式技术

陈润宇

对外经济贸易大学信息学院

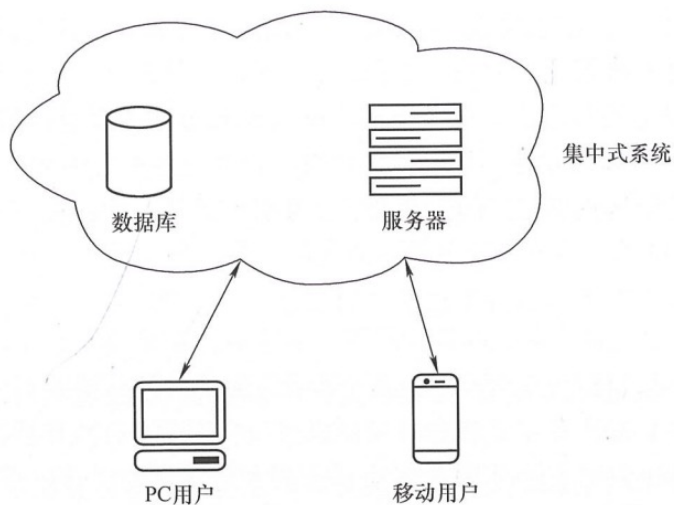


内容提纲

- 分布式系统概述
- 分布式系统理论
- 分布式网络（P2P/对等网络）
- 分布式网络在区块链中的应用

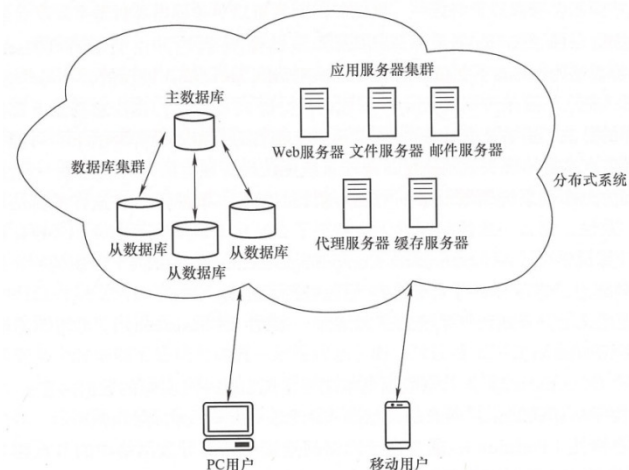
分布式系统概述

- **集中式系统**由一台或者多台计算机组成中心节点，负责管理应用访问的数据存储、计算等资源，增加新的应用需要在中心节点部署更多的计算机。
- 集中式系统一般采用中心化的数据库和服务器，其优点是部署简单、开发运维容易，缺点是可扩展性不足。
- 当集中式系统中心节点进行系统升级更新时，所有应用都需要同步更新。其还存在单点故障问题，如果中心节点出现故障就意味着所有应用都将出现问题。

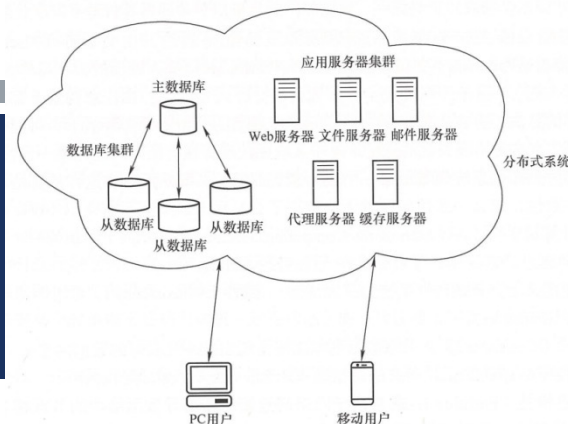


分布式系统概述

- **分布式系统**是由若干独立的计算机节点组成的系统，这些节点可以看成独立的系统组件，通过网络进行连接并在一定范围内有效共享资源，如硬件、软件、数据和服务，节点之间通过传递消息进行协调工作，共同完成系统内的工作任务。
- 分布式系统在数据库和服务器部署上采用集群模式，集群内部存在多个计算机节点。
- 从系统提供服务角度看，分布式系统中的数据库可以由主库和多个独立的从库组成的集群构成，服务器集群根据具体业务应用由部署在不同节点上的服务器组成，用户端应用使用分布式系统提供的服务。
- 该服务可能需要集群内部署的多个服务器节点进行协作，服务器的种类可以根据具体应用进行灵活的扩展。



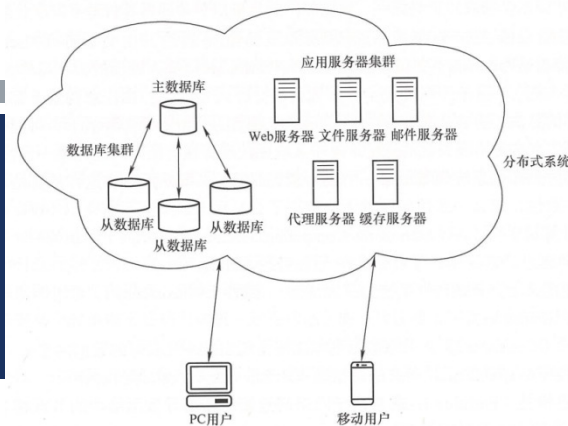
分布式系统概述



■ 分布式系统定义包含两个方面

- 系统内的计算机节点都是独立的，通过网络通信进行协调工作。
- 用户对于分布式系统的访问从功能逻辑上就像访问单个计算机系统一样，并不会感觉到在访问多台分布式计算机系统。
- 分布式系统的基本特征
- 分布性：分布式系统内计算机节点可以分布在不同的位置。
- 可扩展性：分布式系统内节点数量可以根据应用需求进行动态增减，服务器也可以动态部署。
- 对等性：分布式系统没有中心化的控制主机，组成分布式系统的所有计算机节点都是对等的。副本(Replica)是分布式系统最常见的概念之一，指的是分布式系统对于数据和服务的一种冗余的处理方式。数据副本指不同节点上持久化存储同一份数据，当某一个节点上存储的数据丢失时，可以从副本上读取该数据。副本服务指多个节点提供同样的服务，每个节点都有能力接收来自外部的请求并进行相应的处理。
- 并发性：分布式系统中的多个计算机节点通过网络进行连接并在一定范围内有效共享资源，某一时刻这些计算机节点可能会并发地操作一些共享的资源。

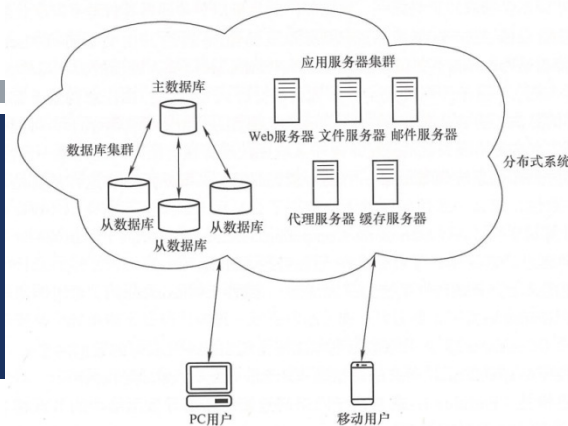
分布式系统概述



■ 分布式系统的一致性

- 指对系统内的所有计算机节点给定一组操作，按照约定的规则协议，节点之间对于操作后的最终处理结果达成某种共同认可的状态。分布式系统的一致性在设计分布式系统时应考虑的最核心问题。
- 由于分布式系统内的计算机节点互相独立，不同计算机节点可能处于不同的地理位置，计算性能也存在差异，对于相同数据任务完成计算耗费的时间无法保证一致。
 - 比如有可能出现少数节点处理较慢，而其他节点必须等待它们处理结束
 - 发生节点临时中断处理等异常情况，如出现节点宕机的情况。
 - 节点之间进行网络通信也有可能因为通信链路故障而导致消息接收延迟，以上这些问题都会影响到分布式系统最终全局状态结果的一致性，需要通过有效的方法解决。
- **分布式系统一致性的目标**
 - 系统在出现上面描述各种故障发生的情况下，依然能正常满足工作的要求，最终系统通过检测和处理，节点依然能达成全局一致性状态。分布式系统的一致性表明，系统本身具有容忍一定数量节点发生错误行为的能力。这些发生错误行为的节点称为故障节点，占整个分布式系统全部节点数量的比例称为分布式系统的容错率。

分布式系统概述



■ 分布式系统的一致性

■ 分布式系统达到一致性状态应满足以下几个基本要求

- 收敛性：一致的结果在有限时间内能完成。收敛性是分布式系统计算机服务可以正常使用的前提。
- 一致性：不同节点最终完成决策的结果是相同的。可以理解为不同节点对计算结果达成的共识。
- 有效性：决策的结果必须是某个节点提出的提案。主要指分布式系统最终一致性是由分布式系统内节点执行的结果。
- 在实际工程实践中，一般会放宽对一致性的要求，采用弱一致性替代强一致性。
 - 强一致性要求节点无论何时进行数据的读取操作，均会返回最新一次写操作后的结果数据，即系统对节点达成一致性结果的同步性要求很高。
 - 而弱一致性一般指在满足一定约束条件下达到最终一致性，即系统可以在未来某个时刻而不是马上达成一致性状态。弱一致性要求中比较典型的是最终一致性(Eventual Consistency)，不保证任意时间点上节点的数据均相同，但需要在经过有限的时间后达到数据上的一致。这实际上可以通过放宽系统的目标要求，从而降低系统实现的难度。

分布式系统理论

■ 数据库的ACID要求

- 在数据库系统中，一个事务是指：由一系列数据库操作组成的一个完整的逻辑过程。例如银行转帐，从原账户扣除金额，以及向目标账户添加金额，这两个数据库操作的总和，构成一个完整的逻辑过程，不可拆分。在数据库管理系统写入或更新资料的过程中，为保证事务是正确可靠的，必须具备四个特性：原子性（atomicity）、一致性（consistency）、独立性（isolation）、持久性（durability），ACID就是这四个基本要素的缩写。
- 关于这四个基本要求的定义，ISO/IEC进行了以下定义
 - Atomicity（原子性）：一个事务中的所有操作，要么全部完成，要么全部不完成，不会结束在中间某个环节。事务在执行过程中发生错误，会被恢复到事务开始前的状态，就像这个事务从来没有执行过一样。
 - Consistency（一致性）：在事务开始之前和事务结束以后，数据库的完整性没有被破坏。这表示写入的资料必须完全符合所有的预设规则，这包含资料的精确度、串联性以及后续数据库可以自发性地完成预定的工作。
 - Isolation（隔离性）：数据库允许多个并发事务同时对其数据进行读写和修改的能力，隔离性可以防止多个事务并发执行时由于交叉执行而导致数据的不一致。事务隔离分为不同级别，包括读未提交、读提交、可重复读和串行化。
 - Durability（持久性）：事务处理结束后，对数据的修改就是永久的，即便系统故障也不会丢失。

分布式系统理论

■ FLP原理

- 是一个关于分布式系统达成共识的重要理论，由费舍尔(Fischer)、林奇(Lynch)、帕特森(Patterson)三位学者在 1985 年提出，并以三人姓氏的首字母作为缩写命名。
- FLP 原理指出，在网络可靠，但允许节点失效的最小化异步模型系统中，不存在一个可以解决一致性问题的确定性共识算法。这个原理实际上说明，对于允许节点失效的情况下，纯粹异步系统无法确保一致性在有限时间内完成，不存在在任何场景下都能实现共识的算法。
- 在分布式系统中，异步指系统中各个节点可能存在较大的时钟差异，同时消息传输时间是任意长的，各节点对消息处理的时间也可能是任意长的，这就导致无法判断某个消息迟迟没有被响应是节点故障还是传输故障。

分布式系统理论

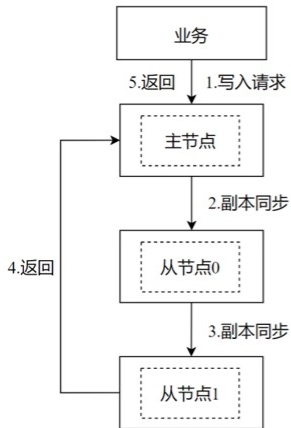
■ CAP理论

- FLP 原理为分布式系统领域带来一个较为悲观的结论，既然不存在这样的算法可以实现分布式系统的一致性，那么一致性问题就无法解决了么？埃里克.布鲁尔(Eric Brewer)在2000 年美国计算机学会(Association for Computing Machinery, ACM)组织的一个研讨会上提出了CAP原理猜想，之后林奇等人对 CAP原理进行了证明。
- CAP原理定义了分布式计算系统的三大特性：数据一致性(Consistency)、系统可用性(Availability)和分区容错性(Partition，也称系统连通性)。
 - 一致性(Consistency):共享数据副本之间呈现出统一且实时的数据内容。
 - 可用性(Availability):所有的数据操作总会在一定时间内得到响应。
 - 分区容错性(Partition):通常由于网络间连接中断而导致网络中的节点相互隔离无法访问时，被隔离的节点仍可正常运行。
- 这三大特性无法同时实现，设计中需要弱化其中某个特性，而保证另外两个特性。对于结果一致性要求不是特别高的应用，可弱化一致性要求，比如延长达成一致性的时间。对于一致性要求高的应用可弱化可用性要求，比如系统发生故障时拒绝服务。大部分时候网络都是可靠的，网络分区出现概率小但很难完全避免，所以实际情况一般弱化分区容错性。

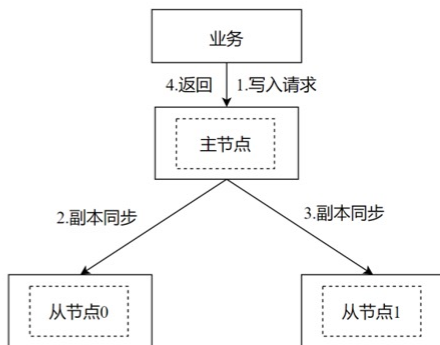
分布式系统理论

■ 分布式数据存储

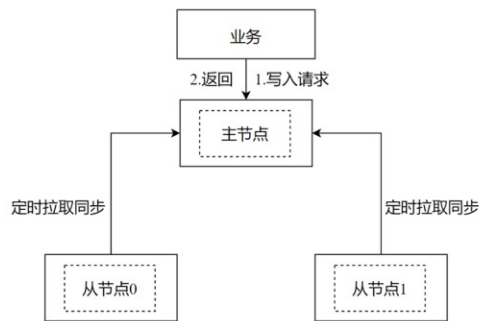
■ CAP中数据一致性(Consistency)的几种实现方法



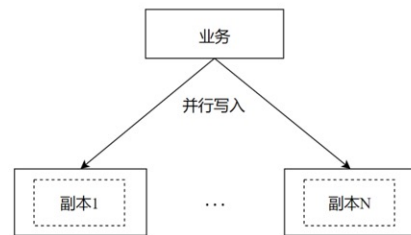
链式写入实现数据一致性



并行写入实现数据一致性[⚠]



定时拉取同步



客户端并行写入实现数据一致性[⚠]

分布式系统理论

■ BASE理论

- BASE理论是eBay架构师Dan Pritchett对大规模分布式系统的实践总结，他首先在ACM上发表文章全面阐述了BASE理论。BASE理论由CAP理论演化而来，它的核心思想是如果一个系统无法做到强一致性，那么可以依据业务自身的特性采取适当的方法使系统达到最终一致性。BASE理论涉及以下三个概念。
 - 基本可用：基本可用是相对于高可用而言的，他表示的是系统在出现故障时还可以使用，只是有一定损失，例如响应速度会变慢，功能上会有损失。
 - 软状态：软状态是相对于原子性处理而言的，原子性处理要求所有的阶段保持一致，做到一致成功或一致失败，而软状态允许在不影响系统可用性的前提下，出现中间不一致的状态。
 - 最终一致性：软状态只是一个中间状态，所以在业务允许的时间范围内需要做到最终一致，这个时间取决于网络延迟、系统负载、数据复制方案设计等因素。最终一致性根据业务场景可分为因果一致性、读己之所写、会话一致性、单调读一致性和单调写一致性。

分布式网络-对等/P2P网络

- 分布式技术在区块链中的应用**主要体现在P2P网络**。P2P网络（Peer-to-Peer，可直译为点对点）也称为**对等网络**，是一种分布式网络，由对等节点在物理网络上形成的一种组网或网络形状，网络的参与者共享他们所拥有的一部分资源，这些共享资源通过网络提供服务 and 内容，能被其他对等节点直接访问而无须经过中间实体。
 - 对等网络实际上是一种在应用层建立的覆盖网络(Overlay Network)，它是在基础的物理网络上利用软件方法构造出的一种逻辑网络。覆盖网络中的节点来自物理网络的主机，通过虚拟链路建立连接，但每一条虚拟链路不一定与一条物理链路对应，可能对应基础网络中的一条或多条物理链路路径。
 - 下图说明了覆盖网络与物理网络的关系，物理网络中的主机可以映射到由虚拟链路建立的覆盖网络，垂直虚线表示了从物理节点到虚拟节点的映射关系。
 - 覆盖网络把特定问题从复杂的底层网络中剥离出来，节点之间通过逻辑链路直接通信，以满足用户特定的应用需求。P2P 网络和常见的 C/S组网模式都是运行在物理网络上的覆盖网络。

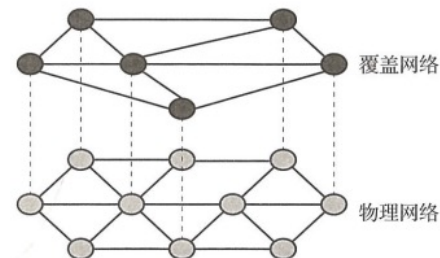


图 4-1 P2P 覆盖网络与物理网络的关系

对等网络发展史

- 早在20 世纪70年代后期，就出现了P2P模式的雏形。例如，1979 年产生的新闻组(USENET)、1984年创建的惠多网(FidoNet)等分布式信息交换系统。由于当时 PC机性能和网络带宽的限制，P2P模式并未得到广泛应用，反而是C/S模式逐渐发展成为互联网上主流的应用模式，许多重要的互联网应用都采用了C/S模式，如浏览器和Web服务器、邮件客户端和邮件服务器等。
- 随着互联网在规模上不断扩展、用户数量持续增加以及多媒体应用的普及，服务器负担越来越重，同时也存在单点故障问题，传统C/S 模式的性能已无法满足用户的需求。在此背景下，人们将目光重新放回被长久忽视的分布式模式上，特别是20 世纪90 年代后期，由于个人计算机的性能获得了极大的提升，P2P技术出现了新一轮的发展高潮。
- 1999年，美国东北波士顿大学一年级新生肖恩·范宁编写了基于 P2P对等模式的 Napster 软件，用于方便音乐爱好者通过 Napster 客户端下载音乐文件(MP3)，同时自己也作为一台服务器，供别人下载。Napster采用的是中央目录服务器的架构，即用户向目录服务器提交文件查询请求，目录服务器返回存放该文件的计算机，用户随后连接到存储该文件的计算机，然后直接从那台计算机上下载文件。这种方法巧妙地应用了互联网体系架构，取得了很好的效果，通过在数百万台计算机上分担下载文件的负载量，Napster 实现了用其他任何方法都无法实现的任务，取得了巨大的成功，最高峰时注册用户达到8000万人。

对等网络发展史

- Napster 的成功带动了其他基于 P2P的文件下载工具的出现，如人们熟悉的努特拉(Gnutella)、比特流(BitTorrent, BT)、电驴(eDonkey)、超网(Overnet)、讯佳普(Skype)等。其中 Gnutella 采用的是纯分布式 P2P 架构，通过洪泛协议在节点之间转发查询请求，实现文件查询，克服了 Napster 中央目录服务器单点故障的问题。2002年布拉姆·科恩发明的比特流协议的特点是下载的人越多，下载速度越快，原因在于每个下载者将已下载的数据提供给其他下载者下载，充分利用了用户的上载带宽。同时，比特流引入了构建结构化P2P网络的分布式哈希表技术，相比洪泛查询技术，DHT（Distributed Hash Table，分布式哈希表）的查询效率得到显著提升。
- 比特币将P2P网络技术应用到区块链领域。区块链并不是为了分享文件而设计的，它主要利用P2P技术构建去中心化的存储框架，存储不可修改和不断增长的交易信息，并实现交易和区块的点对点传播。P2P网络的 DHT技术主要是解决查询效率的问题，比特币之后出现的以太坊采用DHT构建结构化网络，实现高效的节点发现功能。
- 对等网络的发展历史大致可以归结为4个阶段，P2P技术日益成熟后，它在众多领域（包括各种基于区块链的去中心化应用）拥有大量的应用和市场。

表 4-1

对等网络发展历史

阶段	标志性应用
依赖中心索引系统	Napster、Napigator
使用洪泛查询，摆脱中心索引	努特拉
使用分布式哈希表	比特流、超网
区块链相关的分布式存储	比特币、以太坊

对等网络-重要概念

- 节点：也称为网络节点，是指一台计算机或其他具有一个独立地址和数据收发功能的网络设备。节点可以是工作站、个人计算机、服务器和其他连接网络的设备，拥有自己唯一网络地址的设备都是网络节点。
- C/S模式与P2P模式：这两种模式是作为对立面发展起来的，是互联网中两种基本的应用组网模式。C/S模式的应用以服务器为中心，服务器提供服务，所有客户端从服务器获得服务，客户端之间的交互也需要通过服务器进行协调；P2P模式称为对等模式，应用没有专门的服务器，每台计算机既是客户端也是服务器，计算机之间直接建立连接并交换信息。
- 网络拓扑：计算机网络通常由许多的网络节点组成，把这些网络节点用通信线路连接起来，形成一定的几何关系，这就是计算机网络拓扑，简称网络拓扑。对等网络主要发展了4种形式的网络拓扑结构：集中式目录拓扑、纯分布式非结构化拓扑、混合式拓扑和纯分布式结构化拓扑。这些对等网络的拓扑结构各有特点，在不同的特定应用场景都得到了广泛的应用。例如，纯分布式非结构化拓扑和混合式拓扑在比特币、以太坊和超级账本Fabric 等区块链中得到应用，其中以以太坊的节点发现机制还使用了基于分布式哈希表(DHT)技术的结构化拓扑。

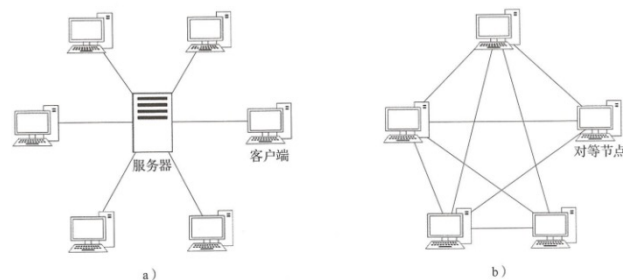


图 4-2 C/S 模式与 P2P 模式的区别
a) C/S 模式 b) P2P 模式

对等网络-重要概念

- 网络协议：是为计算机网络中进行数据交换而建立的规则、标准或约定的集合。对等网络协议主要用于在对等网络中实现节点发现、消息路由和资源搜索，包括非结构化P2P网络路由算法和结构化 P2P网络路由算法，其中非结构化 P2P网络路由算法包括洪泛(Flooding)算法和流言协议；结构化 P2P网络路由算法主要采用DHT技术。
- 分布式哈希表：哈希表(Hash table)是根据键值而直接进行访问的数据结构，它通过把键值映射到表中一个位置来访问记录，以加快查找的速度。这个映射函数叫作哈希函数，存放记录的数组叫作哈希表。分布式哈希表则是一种分布式存储方法，它通过某种方法将键值分散存储在多个节点上，避免了集中存储服务器的单点故障问题。DHT网络中的节点都有一个唯一标识自己的ID，每个资源也有一个唯一ID，资源通常存放在资源ID 和节点ID 相近或者相等的节点上。目前有多种 DHT 技术的实现算法，包括 Chord, Pastry, CAN, Tapestry, Kademlia 等，其中 Kademlia 算法由于简单易用而被广泛使用，以太坊的节点发现机制便采用了Kademlia 算法。
- 对等网络与区块链：尽管对等网络具有去中心化的特点，但是对等网络并不等同于区块链，许多区块链系统都对对等网络进行了一定程度上的改进和创新，以适应自身区块链功能的定位和业务需求。例如，比特币主网络采用对等网络构建了完全去中心化的区块链网络拓扑结构，并使用流言协议实现节点发现、交易和区块链广播功能；以太坊基于Kademlia 算法构建结构化的网络拓扑，实现对等节点的发现机制。

对等网络结构

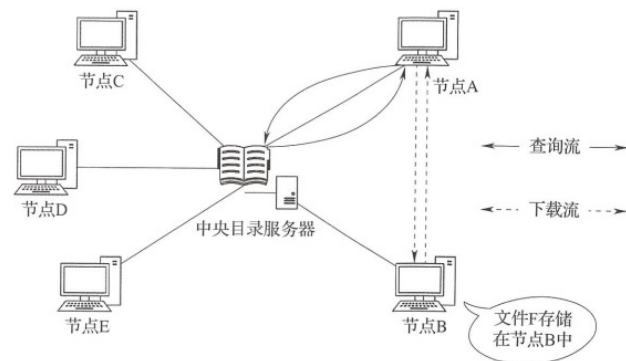


图 4-3 集中式目录结构

- 节点拓扑结构是指分布式系统中各个计算单元之间的物理或逻辑的互联关系，节点之间的拓扑结构是确定系统类型的重要依据。
- 根据网络中节点的连接形式以及资源被索引和定位的方式，可将对等网络分为集中式、纯分布式、混合式和结构化四种不同的网络结构，这也代表着 P2P 技术的 4 个发展阶段。本部分介绍的网络结构主要是指路由查询结构，即不同节点之间如何建立连接通道。
- 一、集中式目录结构
 - 最早出现的 P2P 应用模型，仍然具有中心化的特点。其典型代表是 Napster，网络结构如图所示。集中式目录结构 P2P 网络基于中央目录(索引)服务器，目录服务器保存了所有节点的目录信息(如节点的 IP 地址、保存的文件信息等)，可以对文件查询请求提供快速查询并返回存放有所查找文件的节点地址，随后请求文件的节点，直接与存放文件的节点建立连接以获取文件。
 - 例如在图 4-3 中，节点 A 需要下载文件 F，但不知道文件 F 存放在何处。A 首先向目录服务器提交查询请求，目录服务器向 A 返回文件 F 存放在节点 B 的消息，A 随后与 B 建立连接，从 B 直接下载文件。
 - 集中式目录查询具有结构简单、实现容易、资源发现效率高等特点，且内容传输无须经过中央服务器。由于存在中央目录服务器，集中式目录结构易引发单点故障问题；另外，随着网络规模和查询请求的增加，容易形成传输瓶颈，扩展性较差。因此对小型网络而言，集中式的拓扑模型在管理和控制方面占一定优势，但该模型并不适合大型网络应用。

对等网络结构

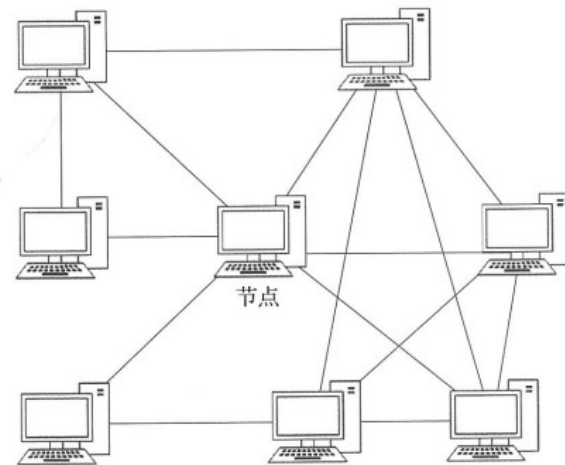


图 4-4 纯分布式非结构化拓扑

■ 二、纯分布式非结构化拓扑

- 纯分布式非结构化拓扑 P2P网络取消了中央的目录服务器，是一种完全分布式非结构化网络。它是采用随机图组织方式而形成的松散网络，每个节点随机接入网络，并与一组邻居节点建立连接，节点之间的内容查询和共享都是通过邻居节点的广播接力传递，自由网（Freenet）和比特币主网结构都是典型的纯分布式结构p2p网络。
- 新节点加入纯分布式P2P网络有多种实现方法，最简单的方法是随机选择一个节点建立邻居关系。
 - 比特币 P2P网络中的新节点则是通过域名解析系统(DomainNameSystem, DNS)种子节点指引快速发现网络中的其他节点。新节点与邻居节点建立连接后，还需要进行全网广播，让整个网络知道该节点的存在。其方式是该节点首先向邻居节点广播，邻居节点收到广播消息后，继续向自己的邻居节点广播，以此类推，从而传播到整个网络，这种广播方法也称为洪泛。
 - 当一个节点想要查询一个文件时，它首先以文件名或者关键字生成一个查询，并把这个查询发送给与它相连的邻居节点，邻居节点如果存在这个文件，则与查询的节点建立连接，如果不存在这个文件，则继续向自己的邻居节点转发这个查询，直到找到文件为止。

对等网络结构

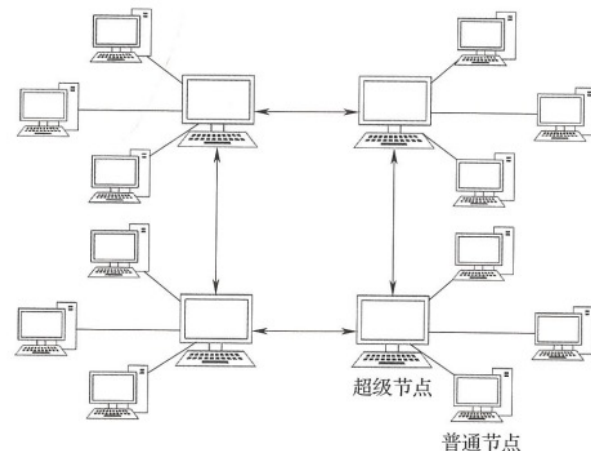
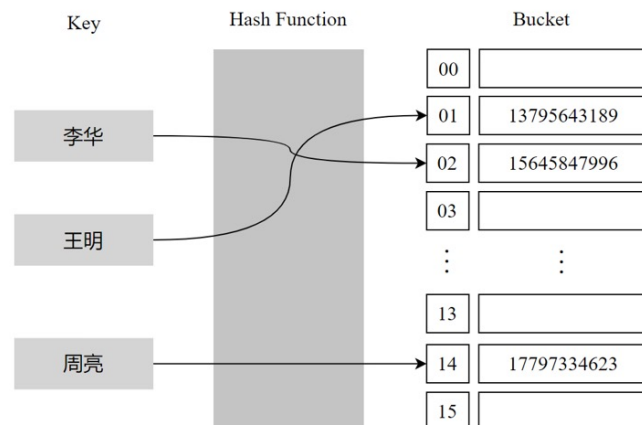


图 4-5 混合模式网络拓扑结构

■ 三、混合模式结构

- 混合模式结构结合了集中式结构和纯分布式非结构化拓扑的优点，网络性能得到优化。混合模式结构如图所示，网络中存在多个超级节点组成分布式网络，而每个超级节点则与多个普通节点组成局部集中式网络。混合模式结构是一个层次式结构，超级节点通常是处理、存储、带宽性能较高的节点，在各个超级节点上存储了系统中其他部分节点的信息，超级节点之间构成一个高速转发层，发现算法仅在超级节点之间转发，超级节点再将查询请求转发给适当的普通节点。
- 新的普通节点要加入网络，首先选择一个超级节点建立连接，该超级节点再将其他超级节点列表推送给新加入的节点，新节点根据超级节点的状态选择适合的节点作为父节点。这种结构的洪泛只发生在超级节点之间，可以减少网络风暴问题。
- 混合式组网结构具有灵活、有效且实现难度小等特点，目前被很多系统所采用，如超级账本区块链平台。

对等网络结构



哈希表示例^[4]

■ 四、纯分布式结构化拓扑

- 结构化P2P网络将节点按某种结构进行有序组织，形成一种逻辑上的结构化网络拓扑，如树状网络或环状网络。由于非结构化P2P网络中的随机搜索具有盲目性，人们开始研究结构化的P2P网络，以提高搜索查询效率。最具代表性的成果是基于分布式哈希表的资源定位算法。
- 在理解分布式哈希表之前，先看看常用的数据结构哈希表。如图所示是一个简单的哈希表，其中键为人名，值为Bucket中的电话号码，可以将Bucket看作一个长度为16的数组。人名通过哈希函数转换为数字，数字作为Bucket数组的下标。下标对应的数组空间用来存储电话号码，最终构成一个简单的哈希表。
- 哈希表所具有的性质为分布式网络环境下快速而准确地路由、定位数据提供了新思路。在哈希表的基础上，分布式哈希表可以通过让每一个节点维护一部分哈希表的方式实现全网节点共同维护全局哈希表的效果。
- 具体而言，分布式哈希表可以将一个关键值的集合分散到所有分布式系统中的节点上，并且可以有效地将信息转送到唯一拥有查询者提供的关键值的节点。这里的节点类似于哈希表中的存储位置。分布式哈希表通常是为了拥有较大节点数量，而且节点常常会加入或离开（如网络断线）的系统而设计的。

对等网络结构

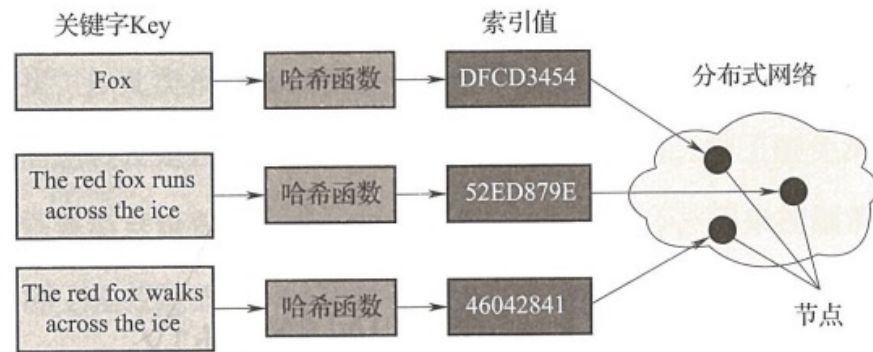


图 4-6 结构化的 P2P 网络结构

■ 四、纯分布式结构化拓扑

- 如图是一个分布式哈希表，数据被哈希之后会得到一个哈希值作为数据的唯一标识，然后将该哈希值根据某种规则和网络中的某个节点关联起来，数据就存放在这个节点中。不同的数据由于被哈希后的值不同，会存储到不同的节点上。这样一来，每个节点都只存储了整体数据的一部分，所有节点一起构成了一个“分布式数据库”。在网络查询数据时，只要知道数据的哈希值，再根据特定的路由规则和存放数据的规则就可以获得相应的节点，最终从节点中获取到需要的数据。
- 基于DHT的结构化P2P网络有以下三个特性，对于完全去中心化P2P网络构建非常重要。
 - 离散性：构成系统的节点并没有任何中心式的协调机制，做到了完全去中心化。
 - 伸缩性：即使有成千上万个节点，系统仍然十分有效。加入的节点越多，网络可提供的服务能力越强。
 - 容错性：即使节点不断地加入、离开或是停止工作，系统仍然能达到一定的可靠性。
- DHT结构的最大问题是维护机制较为复杂，尤其是节点频繁加入/退出造成的网络波动会极大增加 DHT 的维护代价。DHT 所面临的另外一个问题是 DHT 仅支持精确关键词匹配查询，无法支持内容/语义等复杂查询。

对等网络结构

■ 拓扑结构对比

- 在实际应用中，每种拓扑结构的 P2P 网络都有其优缺点，表4-2从可扩展性、可靠性、可维护性、发现算法效率、复杂查询等方面比较了这4种拓扑结构的综合性能。

表 4-2

对等网络拓扑结构对比

拓扑结构	集中式 目录结构	纯分布式 非结构化拓扑	混合模式 结构	纯分布式 结构化拓扑
可扩展性	差	差	中	好
可靠性	差	好	较好	好
可维护性	最好	最好	较好	好
发现算法效率	最高	中	中	高
复杂查询	支持	支持	支持	不支持

对等网络协议

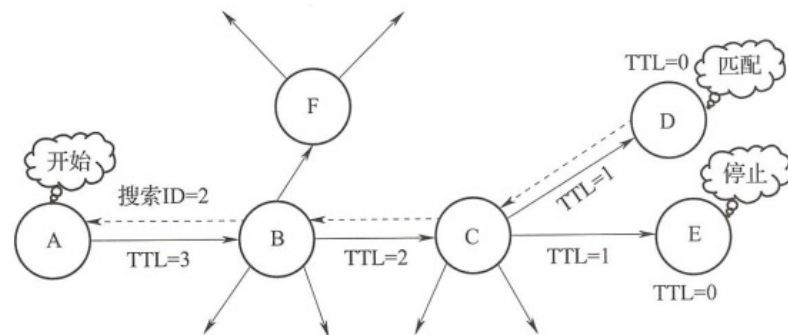


图 4-7 洪泛路由算法

- 对等网络协议也称为网络路由算法，主要用于在对等网络中实现节点发现、消息路由和资源搜索。网络结构的不同，相应的对等网络协议也存在较大的差异。
- 一、非结构化P2P网络路由算法
 - 非结构化P2P网络中没有明确定义的网络拓扑，节点的加入或离开不需要进行网络结构的调整，因此大部分P2P应用系统都是非结构化的，如早期的 Gnutella、比特币和超级账本 Fabric 网络等。在非结构化的 P2P 网络中，资源定位和消息传递主要采用洪泛算法或其优化算法。
 - 洪泛算法
 - 洪泛算法是最早出现的非结构化P2P网络路由算法，路由时具有全网遍历的盲搜索特点。由于每个节点不知道在网络中的哪些节点上有它需要查询的资源，所以当它需要寻找某个资源，将首先把查询信息传递给所有的相邻节点，如果相邻节点有该资源，则返回查询命中消息给查询节点；否则相邻节点继续转发查询消息给各自的相邻节点，继续查找，以此类推，查询消息不断地被转发给更多的节点进行查询。
 - 为了避免消息在网络中循环地传递，为每条消息设置了生命值(Time-to-Live, TTL)，用来控制消息在网络上留存的时间(实际上是控制查询消息的转发次数)。查询节点在生成查询消息时会为其TTL字段设置一个大于0的初始值，节点每转发一次TTL就减1，每个节点会检查要转发消息的TTL值，若为0，则停止转发，抛弃该消息。
 - 优点在于比较简单易于实现。但每次路由都是全网遍历增加了网络的负担，导致搜索的效率不高，网络扩展性差，路由算法容易被攻击。

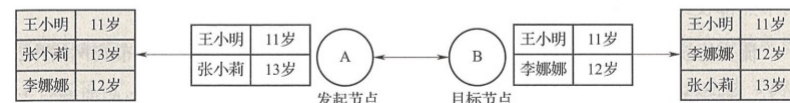
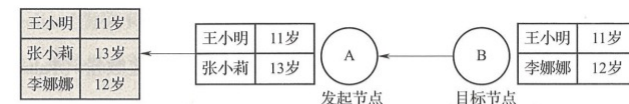
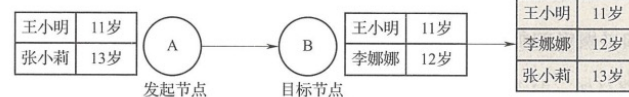
对等网络协议

■ 一、非结构化P2P网络路由算法

■ 流言协议

- 1972年哈伊纳尔等人首次给出了流言问题(电话问题)的描述：有 n 个妇女，每个人都知道一条特有的流言，她们通过电话互相联系；任意两个妇女联系后，互相交流当前自己知道的所有流言；最少需要多少次联系，使得 n 个妇女中的每个人都知道所有流言？这使得对流言问题的研究正式登上历史舞台。1987年，论文《复制数据库维护的疫情流行算法》中最早提出了流言协议，主要用在分布式数据库系统中使节点之间可以高效、可靠地同步数据。
- 流言协议是对洪泛算法的一种改进，通过随机选择部分邻居节点而非全部邻居节点进行广播，从而大大降低了网络的通信量。流言协议主要有两种类型：
 - (1)谣言传播协议(Rumor-Mongering Protocol)。主要思想是当一个节点有了新信息后，该节点变成活跃状态，并周期性地联系其他节点向其发送新信息，直到所有节点都知道该新信息。因为节点之间只是交换新信息，所以大大减低了通信负担。
 - (2)反熵协议(Anti-Entropy Protocol)。每个节点周期性地随机选择其他节点，然后通过互相交换自己的所有数据来消除两者之间的差异。反熵协议非常可靠，但是每次节点两两交换自己的所有数据会带来非常大的通信负担，因此不能频繁使用。
 - 为了在通信代价和可靠性两者之间取得折中，会需要将两种协议结合使用，一般情况采用谣言传播协议，每隔一段时间使用一次反熵协议，以保证信息交换的可靠性。

对等网络协议



一、非结构化P2P网络路由算法

■ 流言协议

■ 目前节点间的信息交换方法主要包含三种

- 推：发起信息交换的节点随机选择联系节点并向其推送自己的信息，一般拥有新信息的节点才会作为发起节点。采用推方式，在信息传播的初期，已感染节点的数目呈指数增长。
- 拉：发起信息交换的节点随机选择联系节点并从对方获取信息，一般无新信息的节点才会作为发起节点。拉方式与推方式相反，在信息传播的初期，感染节点的数目增长缓慢。
- 推拉：推拉是将两者结合起来，发起信息交换的节点向选择的节点发送信息，同时从对方获取数据。

- 比特币节点通过流言协议向分布在不同地区的节点广播交易和新区块；在Fabric中，使用流言在节点间同步新区块。

■ 流言协议的问题

- 消息延迟。节点只会随机向少数几个节点发送消息，消息最终是通过多个轮次的散播而到达全网的，因此使用流言协议会造成不可避免的消息延迟，不适合用在对实时性要求较高的场景下。
- 消息冗余。节点定期随机选择周围节点发送消息，而收到消息的节点也会重复该步骤，因此不可避免地存在消息重复发送给同一节点的情况，造成了消息的冗余，也增加了收到消息的节点的处理压力。此外由于是定期发送，即使收到了消息，节点还会反复收到重复消息，加重了消息的冗余。

对等网络协议

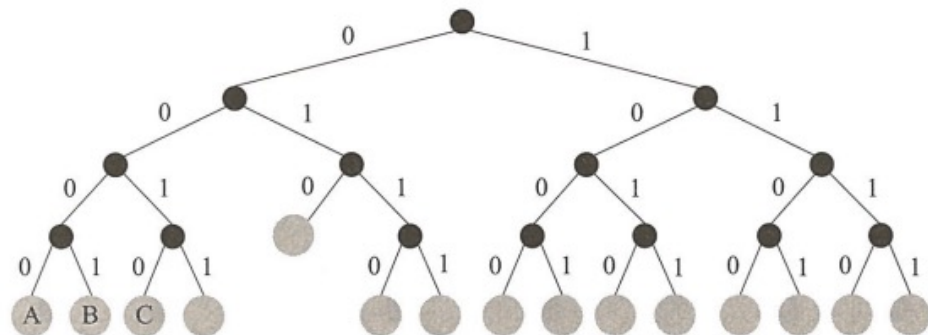


图 4-12 4 位 ID 的网络结构

■ 二、结构化P2P网络路由算法

■ 分布式哈希表

- 分布式哈希表是一种实现分布式存储和资源搜索的技术，用于构建结构化的对等网络。在DHT 中，数据资源以<键，值>对的形式分散保存在节点上，这里值表示文件的存储位置或目录，键是根据文件内容生成的哈希值，整个 DHT 网络维护一张完整的<键，值>文件索引哈希表。
- 用户查询时，根据需要查询的数据资源计算出键和值，然后通过 DHT算法获取键的存储位置，从而快速定位资源的位置。目前，基于 DHT 的典型协议有 Kademlia，Pastry，Tapestry，Chord 等，其中 Kademlia 用于以太坊的节点发现，下文对以太坊中使用的 Kademlia 协议进行详细分析。

■ Kademlia协议理论基础

- Kademlia(简称 Kad)协议是美国纽约大学的梅蒙科夫和马齐耶斯在 2002 年提出的一种 P2P 网络路由算法。Kademlia网络建立了一种树型 DHT 拓扑结构，并使用异或运算来度量节点之间的距离，通过距离分割子树构建路由表，大大提高了路由查询速度。
- 在 Kademlia 网络中，所有节点构成一张虚拟网(或者叫作覆盖网)，这些节点通过160位二进制ID值进行身份标识。根据节点ID可以将网络结构抽象成一棵二叉树，节点的位置由ID的最短前缀唯一确定，0为左边子树，1 为右边子树，每个节点都是二叉树的叶子。

对等网络协议

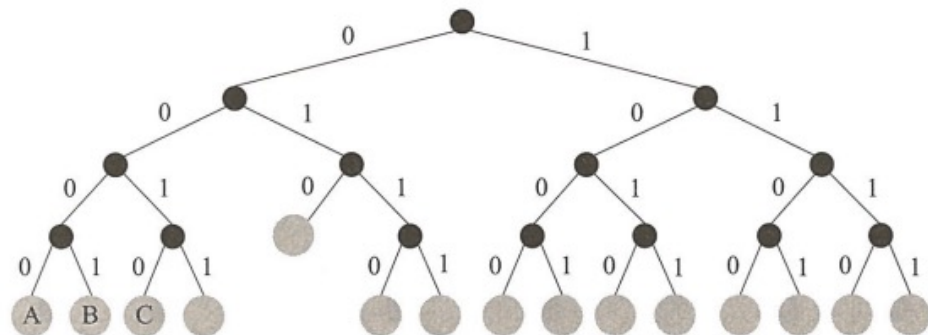


图 4-12 4 位 ID 的网络结构

■ 二、结构化P2P网络路由算法

■ Kademlia协议理论基础

- Kademlia采用异或(XOR) 运算来衡量两个节点间距离(逻辑距离)。对于ID分别为 x , y 的节点, 二者的距离为 $d(x, y) = x \text{ XOR } y$ 。图4-12 中, 节点A 的二进制ID为0000, 节点C的二进制ID为0010, 则A与C的距离 $d = (0000) \text{ XOR } (0010) = (0010) = 2$, 对于异或运算, 有以下数学性质:
 - 非负性: 如果 $x \neq y$, 则 $d(x, y) > 0$;
 - 对称性: 任意 x, y , 有 $d(x, y) = d(y, x)$;
 - 三角不等性: $d(x, y) + d(y, z) \geq d(x, z)$;
 - 单向性: $d(x, y) \text{ (XOR) } d(y, z) = d(x, z)$;
 - 传递性: 对于任意 $a \geq 0, b \geq 0$ 有 $a + b \geq a \text{ (XOR) } b$ 。
- 异或操作具有单向性, 对于任意给定的节点 x 和距离 $\Delta \geq 0$, 总会存在一个精确的节点 y , 使得 $d(x, y) = \Delta$ 。单向性也确保了对于同一个关键字的所有查询都会逐步收敛到同一个路径上, 而不管查询的起始节点位置如何。这样, 只要沿着查询路径上的节点都缓存这个<键, 值>对, 就可以减轻存放热点<键, 值>对的节点的压力, 同时也能够加快查询响应速度。

对等网络协议

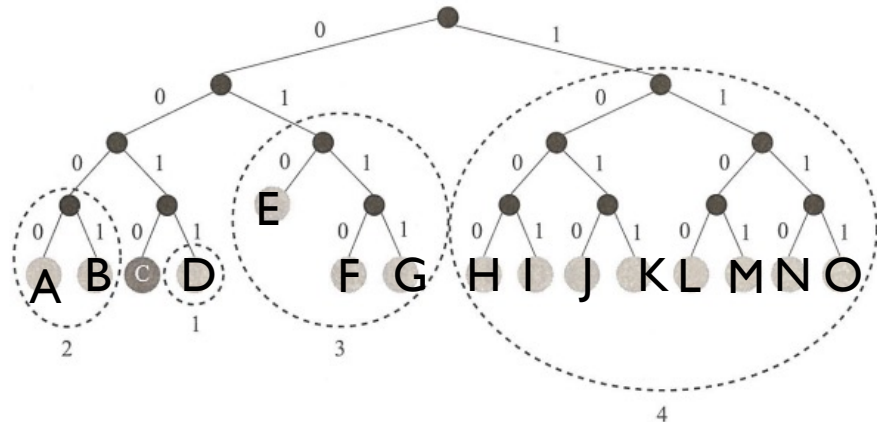


图 4-13 节点 0010 的子树划分

■ 二、结构化P2P网络路由算法

■ Kademlia协议理论基础

- 通过计算距离，每个节点都可以从自己的角度将二叉树拆分为一系列连续的、不包含自己的子树。
- 在图4-13中，从节点C(0010)的角度可以将网络划分为4层子树，如图中虚线圆圈所示，第1层与C节点的前缀位数最多，距离最短，第4层没有共同前缀，距离最远。节点按照自己角度拆分完子树后，一共可以得到 N个子树(N等于二进制ID的长度)，只要知道每个子树里的一个节点就可以实现所有节点的遍历。在实际网络环境中，节点可能会退出网络或者宕机，因此需要记录每个子树里多个节点的路由信息。
- Kademlia算法使用k桶保存每个子树里个节点信息，k 是为平衡系统性能和网络负载设置的常数。对于 160 位ID 的网络，每个节点在完成拆分子树后可以得到160个子树，因此需要维护160个k桶。

对等网络协议

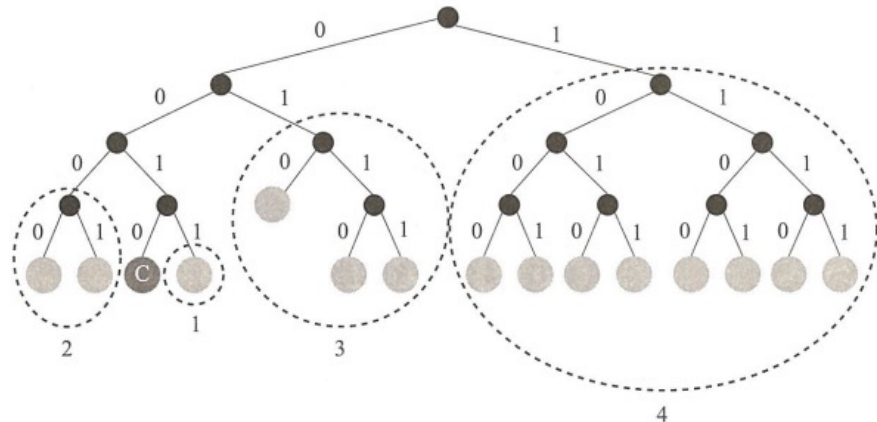


图 4-13 节点 0010 的子树划分

■二、结构化P2P网络路由算法

■ Kademlia协议理论基础

- 任意节点的第 i ($0 \leq i < 160$)个 k 桶记录了与其距离为 $[2^i, 2^{i+1})$ 的节点的信息，表4-3表示 $k=16$ 时，节点每一个 k 桶的覆盖范围和桶中最多能保存的节点数量。当 i 值较小时，由于距离近的子树中包含的节点不多，因此桶中的节点数量较少，对于比较大的 i 值， k 桶节点数可以达到最大值。
- 由于每个 K 桶覆盖距离的范围呈指数关系增长，为了平衡系统性能和网络负载， k 桶中的节点数量通常限制不超过 K 个，表4-3中 $k=16$ ，这就形成了离自己近的节点的信息多，离自己远的节点的信息少，从而可以保证路由查询过程是收敛。经证明，对于一个有 N 个节点的Kademlia网络，最多只需要经过 $\log N$ 步查询，就可以准确定位到目标节点。

表 4-3

k 桶列表

k 桶序号	距离	k 桶所能存储的最大节点数
0	$[2^0, 2^1]$	1
1	$[2^1, 2^2]$	2
2	$[2^2, 2^3]$	4
3	$[2^3, 2^4]$	8
4	$[2^4, 2^5]$	16
5	$[2^5, 2^6]$	16
.....		
159	$[2^{159}, 2^{160}]$	16

对等网络协议

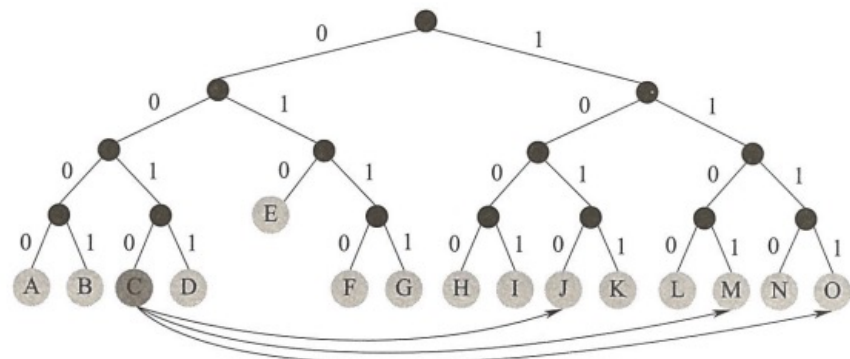


图 4-14 路由过程

■ 二、结构化P2P网络路由算法

■ Kademlia算法查询节点过程

- 如图4-14中节点C (ID为0010) 需要查询目标节点O (ID为1111) 的路由信息。
 - (1) 计算两个节点的距离: $d=(0010)_2 \oplus (1111)_2=1101$ 。
 - (2) 该值落在3号k桶(见表4-4), k桶中没有节点O的信息, 则从k桶内取出最接近1111的个节点, 因此选择节点J(1010), 并向该节点发起查询。
 - (3) 节点J从自己对应的2号k桶中选择节点M(1101)返回给C, 节点 C向该节点发起查询。
 - (4) 节点M从自己对应的1号k桶中选择节点O(1111)返回给C, 节点向该节点发起查询。通过上述查找操作, 节点C得到了目标节点O的路由信息。

表 4-4 节点 0010 的 k 桶信息

k 桶序号	距离	节点路由信息
0	$[2^0, 2^1]$	(IP_addresses [D], port [D], 0011)
1	$[2^1, 2^2]$	(IP_addresses [A], port [A], 0000) (IP_addresses [B], port [B], 0001)
2	$[2^2, 2^3]$	(IP_addresses [G], port [G], 0111) (IP_addresses [F], port [F], 0110)
3	$[2^3, 2^4]$	(IP_addresses [I], port [I], 1001) (IP_addresses [J], port [J], 1010)

对等网络协议

■ 二、结构化P2P网络路由算法

■ Kademlia算法查询节点过程

- 为了保证k桶中的节点是在线的，节点必须对k桶进行更新。当节点x收到从节点y发出的一个远程过程调用(Remote Procedure Call, RPC)消息时，提取节点y的<键，值>里的“值”更新节点x的k桶中节点y对应的路由信息，具体步骤如下：
 - (1)计算节点x和节点y的距离： $d(x, y) = x \text{ XOR } y$ ，其中x和y分别代表节点x和节点y的ID值。
 - (2)对节点x的第 $\lceil \log_2 d \rceil$ 个桶进行更新操作。
 - (3)如果节点y的“值”已在k桶中，则把它移到k桶的尾部。
 - (4)如果节点y的“值”没有被记录在k桶中，则执行如下操作：①如果桶未满，则直接把节点y的“值”插入k桶尾部；②如果k桶已满，则选择k桶头部的记录项(假如是节点z)并PING节点z，如果节点z未响应，则从k桶中移除节点z的“值”，并把节点y的“值”插入k桶尾部；如果节点z响应，则把节点z的“值”移到桶尾部，同时忽略节点y的“值”。

对等网络协议

■ 二、结构化P2P网络路由算法

■ Kademlia算法查询节点过程

- k 桶的更新机制非常高效地实现了一种更新最近看到节点的策略，即在线时间长的节点具有较高的可能性继续保留在k桶列表中。通过把在线时间长的节点留在桶里，明显增加了k桶中节点在下一时间段仍然在线的概率，这对网络的稳定性和减少网络维护成本(不需要频繁构建节点的路由表)带来很大好处。
- 这种机制的另一个好处是能在一定程度上防御拒绝服务(Denial of Service, DoS)攻击，因为只有当老节点失效后，才会更新K桶的信息，这就避免了通过新节点的加入来洪泛路由信息。
- Kademlia 协议以独特的异或运算来计算节点间的距离，提高了路由和搜索的速度，系统具有较好的稳定性、可扩展性和负载平衡性。但异或运算也使网络节点经过哈希运算后的物理位置信息被破坏，来自同一个子网的多个节点的逻辑距离可能很远，路由过程要跨越多个广域网节点，导致应用系统响应时间长，降低了网络速度。

P2P网络在区块链1.0中的应用

- 不管是区块链1.0或是区块链2.0，区块链在其**网络拓扑构建、节点发现和节点间信息传播环节都应用了P2P网络技术**。虽然不同的区块链平台实现了不同的 P2P网络协议，其网络模型可能不同，但是基本原理是相似的。对于区块链P2P网络而言，节点发现环节是构建网络的开始。
- 1. P2P网络结构
 - 区块链1.0主要实现加密货币应用，典型代表是比特币网络的底层技术。比特币网络由大量运行比特币协议的节点组成，采用了混合式 P2P网络路由结构。网络中的节点主要有4 大功能：钱包(Wallet)、矿工(Miner)、完整区块链数据库(Full Blockchain)和网络路由(NetworkRouting)。对于节点而言，网络路由功能是必不可少的，其他功能是可选的。如图4-15所示，一般而言，只有比特币核心节点才会包含全部的4大功能。

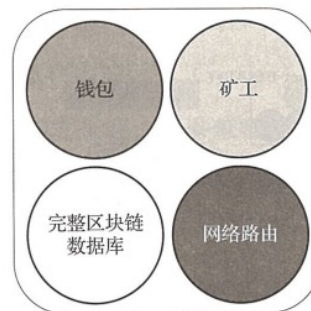


图 4-15 网络核心节点的 4 大功能

P2P网络在区块链

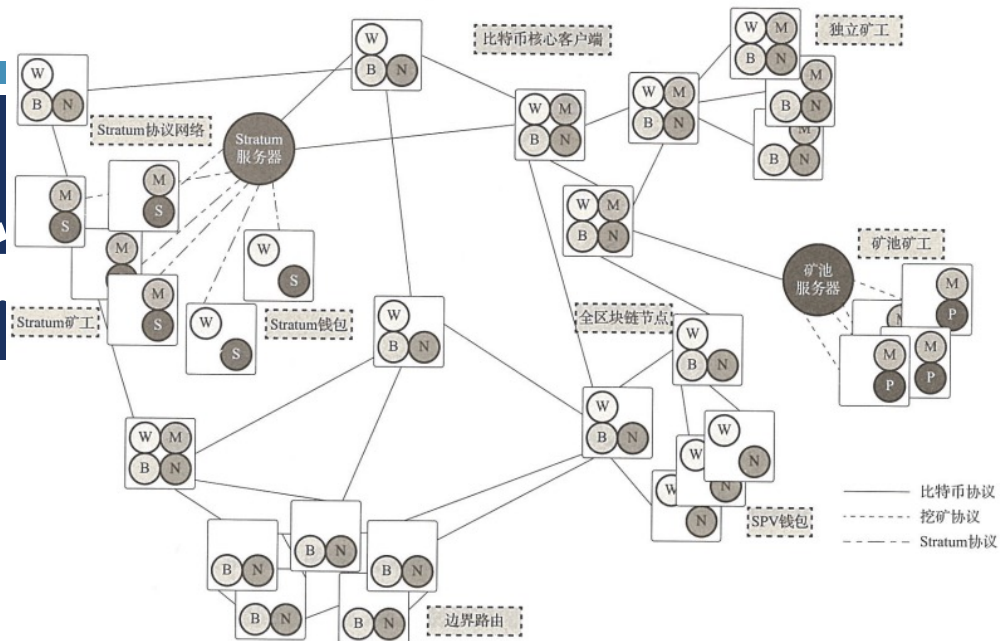


图 4-16 典型的区块链 1.0 网络结构

■ 1. P2P网络结构

- 比特币网络中所有节点都会参与校验、连接。部分节点会包含完整的区块链数据库，即所有的交易数据和打包的区块，这类节点也被称为全节点。另外一些节点只储存区块链数据库的一部分数据，一般是区块头数据，而非全部的交易数据，这类节点会通过“简化交易验证(Simplified Payment Verification, SPV)”的方式完成交易校验，这类节点也被称为 SPV 节点或轻节点。
- 钱包功能一般是个人计算机或手机客户端的功能，即通过钱包查看账户余额，管理钱包地址、私钥以及发起交易等。
- 部分矿工节点同时也是全节点，被称为独立矿工。与其他矿工节点一同连接到矿池进行集体挖矿的节点被称为矿池矿工。矿池是一个局部的集中式挖矿网络，中心节点是一个矿池服务器，其他矿池矿工全部连接到矿池服务器。矿池矿工和矿池服务器之间的通信使用矿池挖矿协议，而矿池服务器作为一个全节点再与其他比特币节点使用主网络的比特币协议进行通信。
- 在整个比特币网络中，除了使用比特币协议作为通信协议的节点所构成的主网络，也存在很多扩展网络，包括上面提到的矿池网络。不同的矿池网络可能还会使用不同的矿池挖矿协议，目前主流的矿池协议是阶层(Stratum)协议，该协议除了支持挖矿节点，也支持瘦客户端钱包。

P2P网络在区块链1.0中的应用

■ 2. 节点发现

- 节点发现是新节点加入区块链网络的第一步。新节点首先要发现节点，建立邻接关系，然后才可以实现交易、验证等功能。以比特币网络为例，新节点可采用如下两种方式发现节点：
 - (1)利用 DNS种子节点。比特币客户端的列表中记录了一些长期稳定运行的 DNS节点，这些节点称为DNS 种子节点，种子节点的地址被编码到比特币源码中，比特币核心客户端包含5个不同的 DNS种子节点。通过种子节点，新节点可以快速地发现网络中的其他节点。用户可通过默认开启的“switch-dnsseed”选项指定是否使用种子节点。
 - (2)节点引荐。用户通过“-seednode”选项可以指定一个节点的IP地址，新节点启动后与该节点建立连接，将该节点作为 DNS 种子节点，在引荐信息形成之后断开与该节点连接，并与新发现的节点连接，下面是节点引荐的具体过程。新节点通常在8333端口采用TCP协议与已知的对等节点建立连接。在建立连接时，节点发送一条包含基本认证内容的版本(version)消息开始“握手”通信，收到版本消息的节点检查自己是否与之兼容，如果兼容，则返回版本确认(verack)消息进行确认并建立连接。有时节点间需要互换连接，接收端也需给发起端发送版本消息。初始化握手过程如图4-17所示。一旦建立连接，新节点发送一条包含自身IP地址的 addr 消息给已连接的节点，节点收到此消息后，继续将该消息转发给各自的连接节点，使网络中更多的节点接收到新节点的消息，保证连接更加稳定。此外，新节点可以向相邻节点发送getaddr 消息，请求返回其已知节点的IP地址列表，从而找到更多可连接的节点，如图4-18所示。

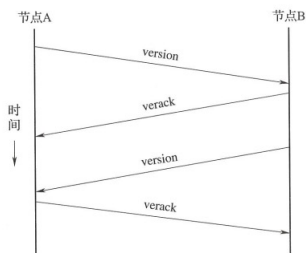


图 4-17 对等节点之间的初始化握手

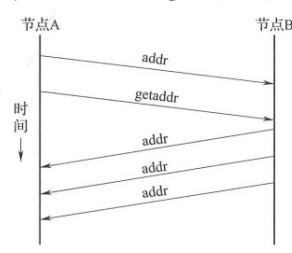


图 4-18 地址传播过程

P2P网络在区块链1.0中的应用

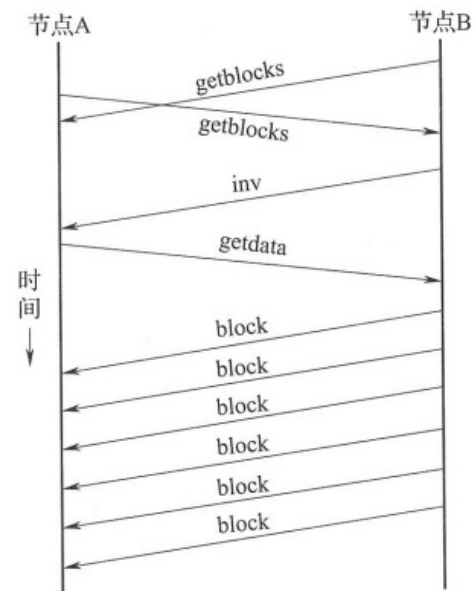


图 4-19 同步区块链数据的过程

■ 2. 节点发现

■ 区块链同步

- 全节点存储并维护完整的区块链副本，可以独立校验交易和区块。新节点刚加入区块链网络时，仅包含静态植入客户端中的0号区块(创世区块)，需要下载从0号区块到最新区块的全部区块后，才能成为全节点，独立参与维护区块链和创建新区块。
- 以比特币区块链为例，同步区块链的过程以发送version 消息开始，通过版本消息中 BestHeight 字段可知道双方节点的区块高度，通过交换 getblocks消息可获取顶部区块的哈希值，从而可准确比较节点所存储区块链的长度。
- 拥有更长链的节点判别其他节点需要“补充”的区块后，开始分批发送区块(500个区块为一批)，通过inv 消息将第一批区块清单广播出去。缺少区块的节点通过发出一系列 getdata 消息，请求得到完整的区块数据，并使用inv消息中的哈希值确认区块的正确性。
- 例如，假设一个只有创世块的新节点收到来自其他节点的inv消息，便向与之相连的所有节点请求区块，并通过分摊工作量的方式减轻单一节点的压力。如果节点需要更新大量区块，需在上一请求完成后才可发送新请求，从而控制更新速度减小网络压力。被接收的区块不断添加至区块链中，直到该节点与全网络完成同步为止。当节点离线后重新返回区块链网络时，会与所连接节点进行区块比较，发送getblocks 消息下载缺失的区块。

P2P网络在以太坊中的应用

- 和比特币一样，以太坊的节点也具备钱包、矿工、完整区块链数据库和网络路由4大功能，存在很多不同类型的节点，除了主网络之外也同样存在很多扩展网络。
- 然而以太坊采用了有结构的 P2P底层网络，主要包括两个协议:节点发现协议(discv4)和节点通信协议(rlpv)。其中 discv4 协议是一种类 Kademlia 协议，主要用于以太坊节点发现环节。虽然 Kademlia是为了在 P2P网络中有效地定位和存储内容而设计的，但以太坊的P2P网络只用它来发现新的对等节点。
- 以太坊节点发现
 - 在以太坊的discv4网络中，节点通信采用UDP模式，并使用节点通信协议rlpv 进行对等通信。其主要通过以下4种报文命令实现节点探测、响应、查询以及应答等功能。
 - Ping:用于探测节点是否在线，Ping 发送后，若15 s 内没有收到 Pong 响应，将自动重发 Ping，最多发送三次，三次都没有收到响应，则相应的节点状态将从 discovered变为 dead，将Evict Candidate 变为 NonActive。
 - Pong:用于响应 Ping 报文，节点一旦接收到 Ping 消息，马上发送 Pong 消息，并将对方节点的状态改为 Alive 或者 Active。
 - FindNode:用于查找与Target节点异或距离最近的其他节点。
 - Neighbors:用于响应 FindNode 报文，从 k 桶里面查找最接近目标标识符的节点，回传找到的邻居节点的列表。

P2P网络在以太坊中的应用

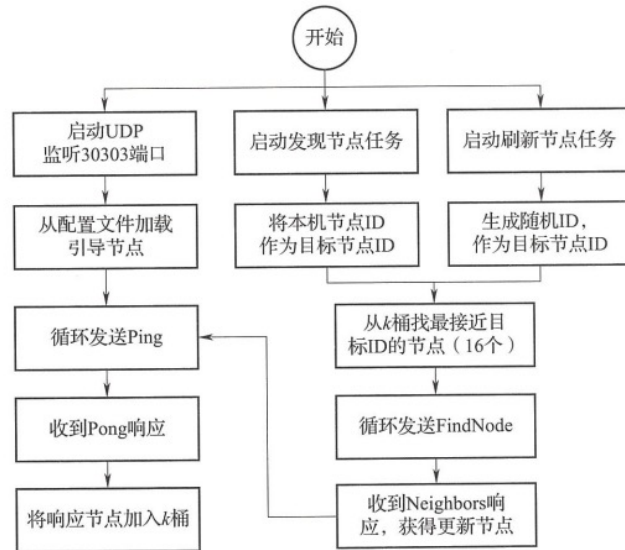


图 4-21 以太坊节点发现过程

■ 在以太坊中发现其他节点

- 新节点要加入以太坊网络，客户端Geth需要通过种子节点引导区发现其他节点。发现流程：
 - (1)节点第一次启动时，随机生成本机节点ID，记为LocalID，生成后该ID将固定不变，同时打开30303端口，监听节点发送协议网络(使用UDP)。
 - (2)从配置文件加载引导节点，向这些节点循环发送Ping报文，在线的引导节点将响应Pong报文，将响应的引导节点加入某一k桶。
 - (3)节点在启动UDP监听的同时，启动了另外两个任务：节点发现任务和节点刷新任务。
 - (4)节点发现任务每30s循环一次，主动寻找邻居节点，以保证桶的节点是满的。每次循环搜索8次，每次搜索以 Local ID 为目标节点 ID，记为 Target ID，从桶中获取距离Target ID 最近的 16个节点，循环向这16个节点发送FindNode 报文(包含Target ID)。
 - 节点发现任务采用非定期或定期循环方式，当收到 FindNode 报文时会立即启动一次节点发现任务，定期循环通过定时器实现，至多等待1800。(事实上，每次以太坊源码迭代都有可能修改节点发现等待时间，最早的设计是3600s)，将会主动寻找邻居节点，以保证尽可能填满k桶中存储的节点信息。每次循环搜索8次，每次搜索以Local ID为目标节点的Target ID，从 k桶中获取距离Target ID 最近的 16个节点，循环向该 16个节点发送FindNode 报文(包含Target ID)。

P2P网络在以太坊中的应用

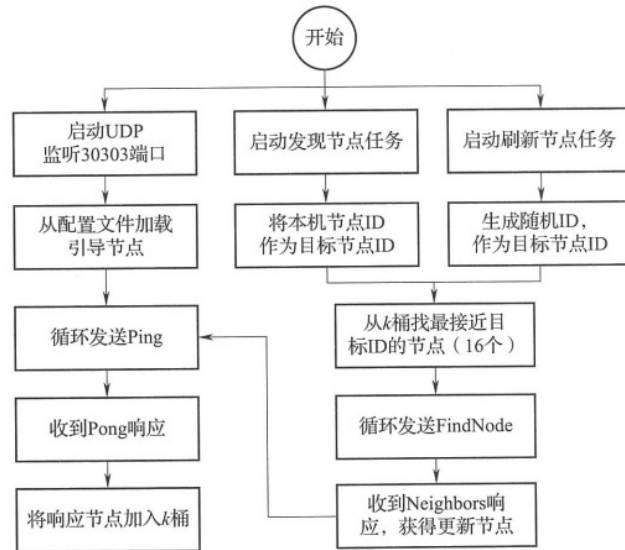


图 4-21 以太坊节点发现过程

■ 在以太坊中发现其他节点

- 新节点要加入以太坊网络，客户端Geth需要通过种子节点引导区发现其他节点。发现流程：
 - (5)收到 FindNode 报文的节点也以Target ID为目标，从自己的桶中找出距离最近的16个节点，然后回传Neighbors报文。
 - (6)节点收到 Neighbors后，从报文取出新发现的节点，向新节点循环发送 Ping报文，并将响应的节点加入桶。经过8次循环搜索，所查找的节点均在距离上向Target ID 收敛，桶中存储的是不断靠近Target ID 的节点。
 - (7)节点刷新任务与节点发现任务类似，但有两点不同:①刷新任务的TargetID不是LocalID，而是随机生成的节点ID;②刷新任务的刷新速度更快，每 7.2 s 循环一次。
 - 节点刷新任务采用了定期刷新的方式，但与节点发现任务不同点在于:①刷新任务的Target ID 不是 Local ID，而是随机生成的节点 ID;②刷新任务的定时器等待时间更短，至多等待 10s。
 - (8)通过上述步骤不断发现和刷新节点，当前节点会找到越来越多的邻居节点，组成k 桶路由表。

P2P区块链应用总结

- 不同结构的 P2P网络，会有不同的优点和缺点。比特币网络的结构明显容易理解，实现起来也相对容易得多，而以太坊网络引入了异或距离、二叉前缀树、h桶等，结构上复杂不少，但在节点路由上会比比特币快很多。
- 不管是比特币还是以太坊，其实都只是一种或多种协议的集合，不同节点其实可以用不同的语言实现。例如，比特币就有用 C++实现的 BitcoinCore，还有用Java 实现的 Bitcoinj;以太坊也有用 Go 语言实现的 go-ethereum，也有用 C++实现的 go-ethereum，还有用Java 实现的Ethereum (J)。